

Analyse Comportementale Clientèle Retail

Atelier Pratique Machine Learning — Module GL2

Préparée par : Chahd Gharbi

Encadrante : Mme Fadoua Drira

Exploration → Préparation → Modélisation → Évaluation → Déploiement

Table de matières

01 Introduction et Contexte

Problématique métier, objectifs, jeu de données

02 Exploration des données

Chargement, diagnostic qualité, corrélations

03 Traitement des données

Réduction dimensionnelle, segmentation clientèle

04 Modélisation : ACP et K-Means

Réduction dimensionnelle, segmentation clientèle

05 Modélisation et Classification

Random Forest, XGBoost, Stacking, features importance

06 Régression et Évaluation

Prédiction montant dépensé, comparaison modèles

07 Déploiement

Application Flask

08 Conclusion

recommandations, perspectives

01-Introduction

Le secteur du e-commerce retail génère de grandes quantités de données clients. Sachant que fidéliser un client existant coûte 5 à 7 fois moins cher que d'en acquérir un nouveau, anticiper le churn devient un enjeu stratégique essentiel.

01-contexte et problématique

Problématique Métier

Le churn (départ client) est le défi n°1 de l'e-commerce

Acquérir un nouveau client coûte 5 à 7× plus cher que retenir un client existant.

Question centrale :

Comment identifier les clients à risque élevé de churn à partir de leurs données comportementales et transactionnelles, afin de déclencher des actions de fidélisation ciblées ?

Objectifs complémentaires ?

- Segmenter la clientèle (K-Means)
- Prédire le churn (classification)
- Estimer la valeur client (régression)

02-Exploration et analyse des données

Dataset Retail

4 372 Clients

52 Variables

67% Fidèles (0)

33% Churners (1)

52 variables réparties en

- 18 Variables catégorielles
- 34 variables numériques

02-Exploration et analyse des données

Valeurs manquantes

Age, AvgDaysBetweenPurchases
(30% pour Age)

✓ Imputation médiane (train uniquement)

Valeurs aberrantes

SupportTickets (−1, 999),
SatisfactionScore (−1, 99) — 5–8%

✓ Remplacement NaN → imputation

Formats inconsistants

RegistrationDate (UK, ISO, US) —
100% des lignes

✓ `pd.to_datetime` multi-format

Feature sans variance

NewsletterSubscribed (toujours
'Yes') — constante

✓ Suppression immédiate

02-Exploration et analyse des données

Données brutes

LastLoginIP (adresse IP) — 100%

✓ Conversion booléen IsPrivateIP

Multicolinéarité

MonetaryMin ↔ MonetaryStd
r=0.97, TotalQty ↔ AvgQty r=0.91

✓ Suppression par VIF > 10

Déséquilibre classes

Churn : 33% churners / 67%
fidèles

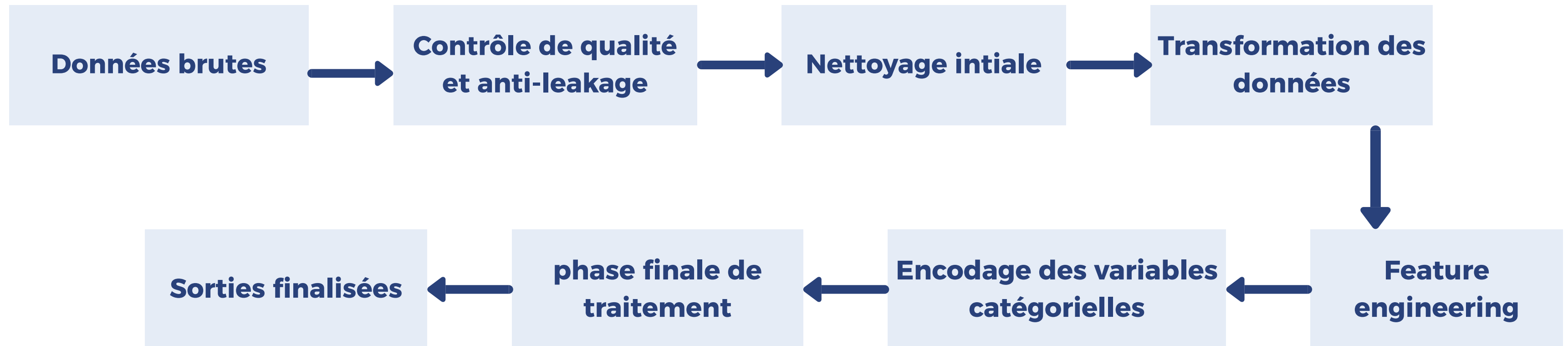
✓ SMOTE (sampling_strategy=0.5)

Corrélations clés

- MonetaryMin ↔ MonetaryStd: r = 0,97
- TotalQuantity ↔ AvgQtyPerTrans: r = 0,91
- UniqueProducts ↔ UniqueDesc: r = 0,88

03-Préparation des données

étapes de traitements



03-Préparation des données

Feature Engineering (3 nouvelles variables)

$\text{MonetaryPerDay} = \text{MonetaryTotal} / (\text{Recency} + 1)$

→ *Dépense quotidienne moyenne*

$\text{AvgBasketValue} = \text{MonetaryTotal} / \text{Frequency}$

→ *Valeur moyenne du panier*

$\text{TenureRatio} = \text{Recency} / (\text{CustomerTenureDays} + 1)$

→ *Ratio ancienneté / activité récente*

03-Préparation des données

Data Leakage : Détection et Correction

Symptôme : Random Forest → accuracy 100% (trop parfait!)

Après correction : 94,5% — réaliste et généralisable

Colonnes supprimées :

- Dérivées du churn : ChurnRiskCategory, CustomerType, LoyaltyLevel, SpendingCategory, RFMSegment, AccountStatus
- Post-churn : ReturnRatio, NegQtyCount, ZeroPriceCount
- Corrélation directe : Recency ($r=0,86$), TenureRatio

03-Préparation des données

Encodage des Variables Catégorielles

- Ordinal : AgeCategory, BasketSizeCategory, PreferredTimeOfDay
- One-Hot : FavoriteSeason, Region, WeekendPreference, Gender...
- One-Hot différé : Country (37+ modalités) avec `handle_unknown='ignore'`

Split Train/Test & Normalisation

- **87 features après encodages One-Hot**
- Split stratifié 80/20 : 3 497 train / 875 test (`random_state=42`)
- Imputation : SimpleImputer (médiane) ajusté sur X_train uniquement
- Standardisation : StandardScaler ajusté sur X_train seulement

04-ACP : Réduction Dimensionnelle

Pourquoi l'ACP ?

- 87 features après encodages

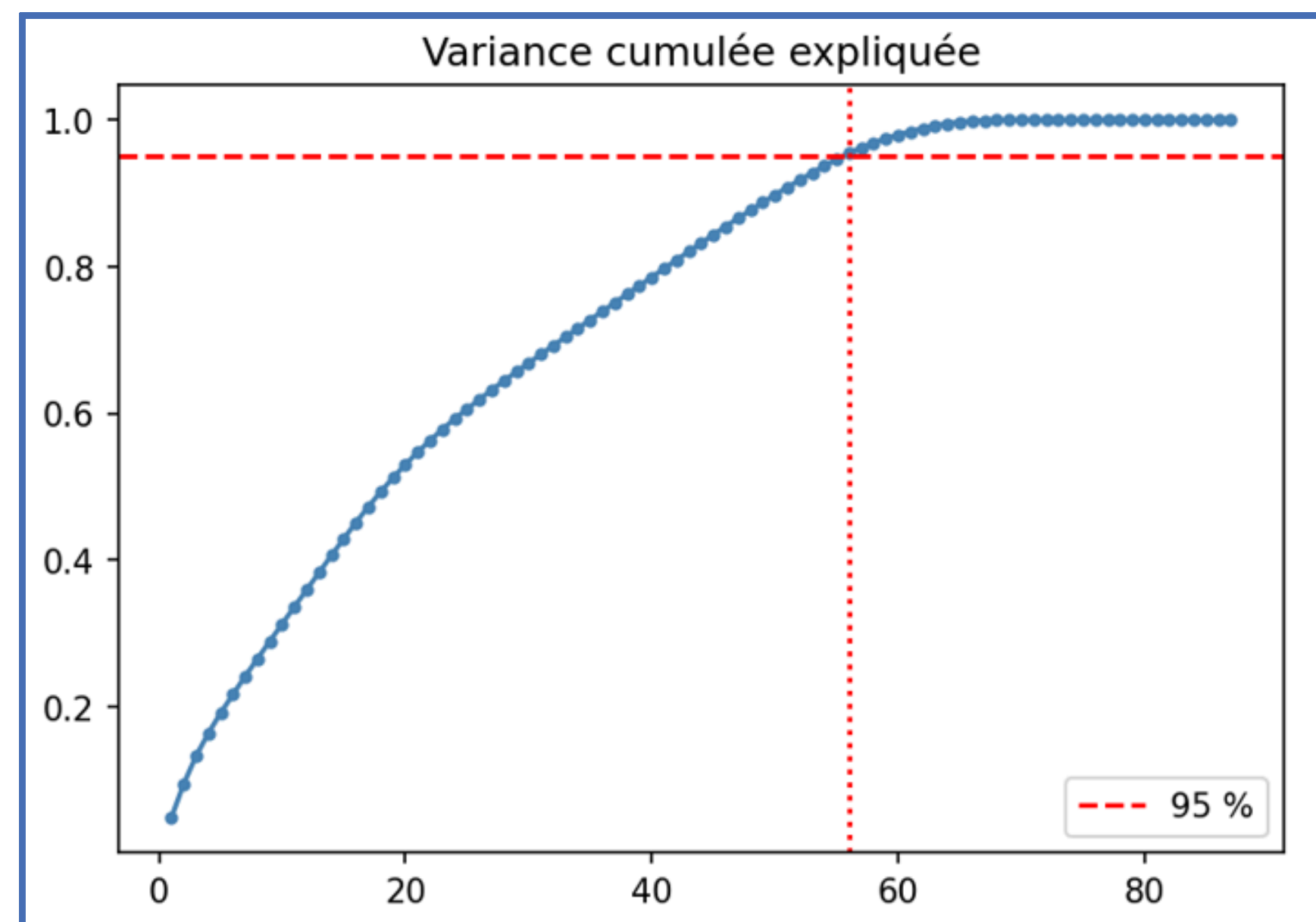
Risques sans ACP :

- Sur-apprentissage (malédiction de la dimensionnalité)
- Temps de calcul accru
- Difficultés de visualisation

Résultats ACP :

- 56 composantes → 95,5% de variance conservée
- Appliquée sur X_train_scaled
- Ces 56 PC alimentent le K-Means
- Score silhouette sans ACP : 0,038
- **Score silhouette avec ACP : 0,086 (+127%)**

Variance cumulée expliquée



← 95,5% atteint avec 56 composantes

04-K-Means : Segmentation Clientèle



Clients Premium

1 453 — 33,2%

Haute fréquence,
montant élevé,
faible récence.
Clients les plus
fidèles et
rentables.



Clients Réguliers

884 — 20,2%

Fréquence et
montant modérés.
Clients stables à
consolider.



Clients Occasionnels

928 — 21,2%

Fréquence faible,
achats
sporadiques. Fort
potentiel de
réactivation.



Clients Inactifs

232 — 5,3%

Très longue
récence, aucune
activité récente.
Risque élevé de
churn.

05-Random Forest : Classification Supervisée

Principe :

Ensemble de T arbres de décision entraînés sur sous-ensembles aléatoires (bagging).

Prédiction = vote majoritaire $\hat{y} = \text{mode}(h_1(x), \dots, h_T(x))$

Pipeline :

SMOTE → RandomizedSearchCV (60 combos, 5 plis) → Seuil optimal

Meilleurs hyperparamètres :

- `n_estimators = 1000`
- `max_depth = 40`
- `min_samples_leaf = 3`
- `max_features = 0,7`
- Seuil optimal = 0,47 (F1 = 0,929)

05-Random Forest : Classification Supervisée

0,985

AUC-ROC

94,5%

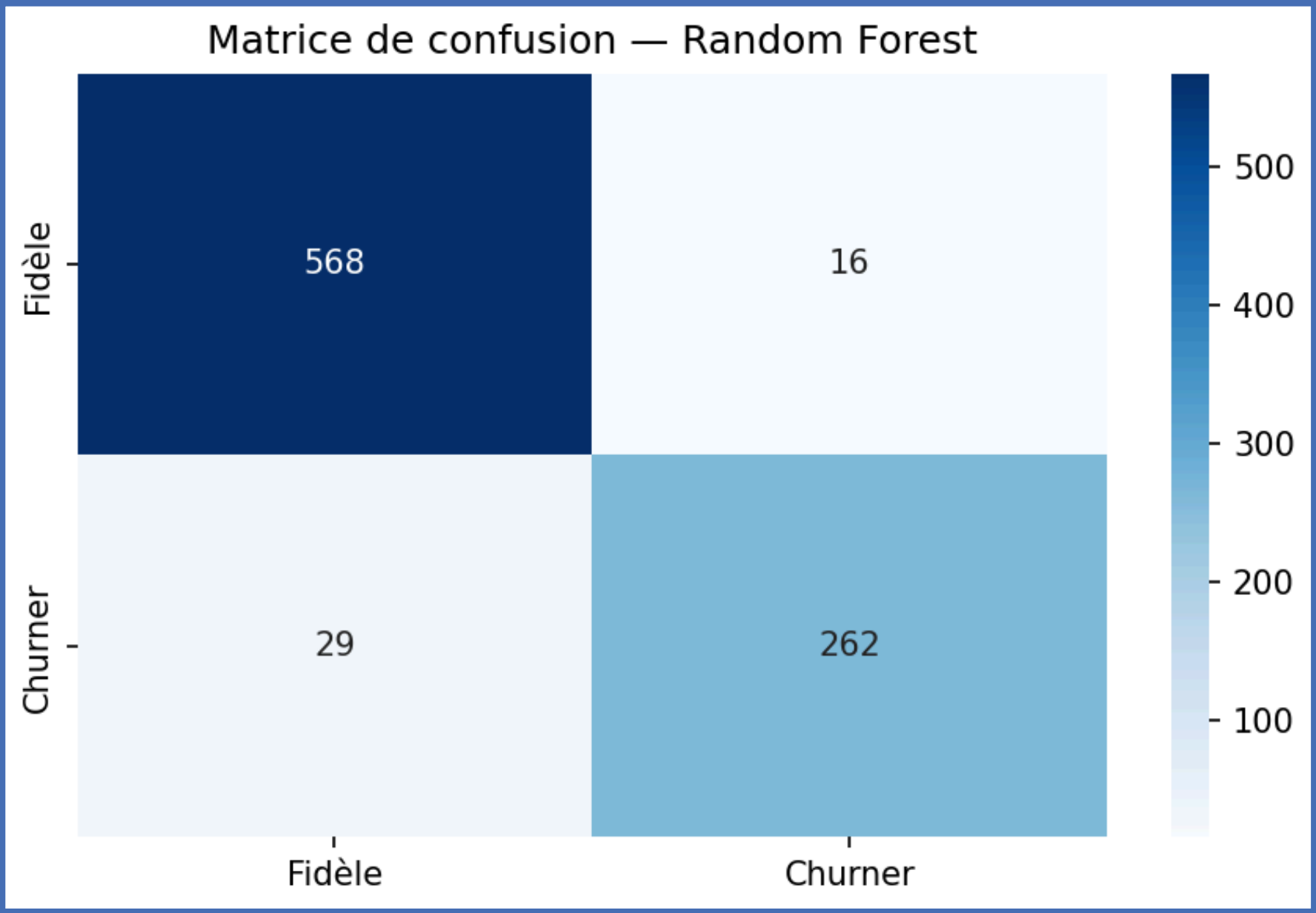
Accuracy

0,92

F1 Churner

0,47

Seuil



XGBoost

Principe :

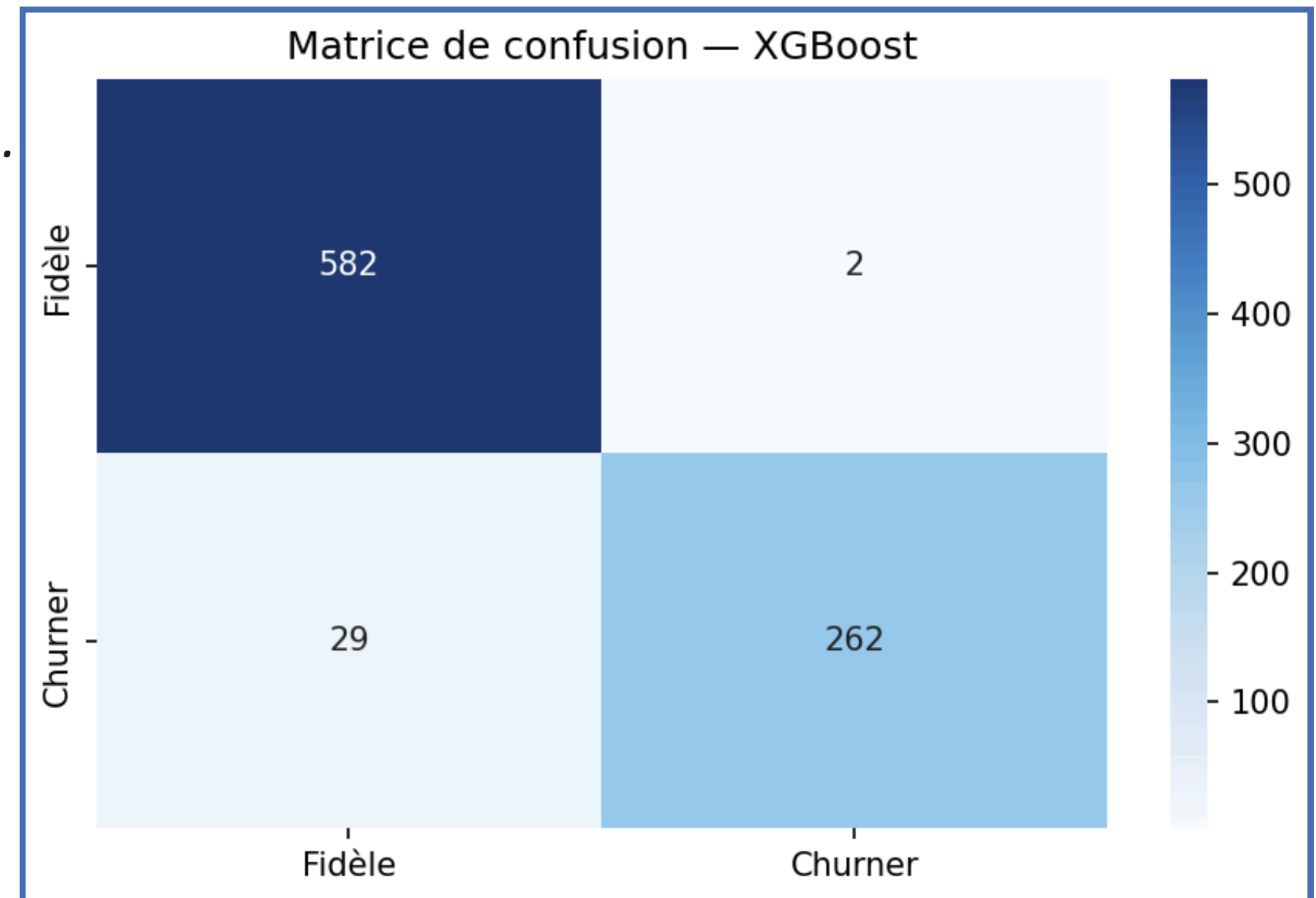
chaque arbre corrige les erreurs du précédent via minimisation d'une fonction de perte régularisée (L1/L2).

Hyperparamètres (GridSearchCV, 5 plis) :

- **n_estimators = 700**
- **max_depth = 5**
- **learning_rate = 0,03**
- **Seuil optimal = 0,60**

Résultats :

- **Fidèle (0) : P=0,96 | R=0,99 | F1=0,98**
- **Churner (1) : P=0,99 | R=0,92 | F1=0,96**
- **Accuracy = 97,1% | AUC = 0,994**



Stacking

Architecture en 3 niveaux :

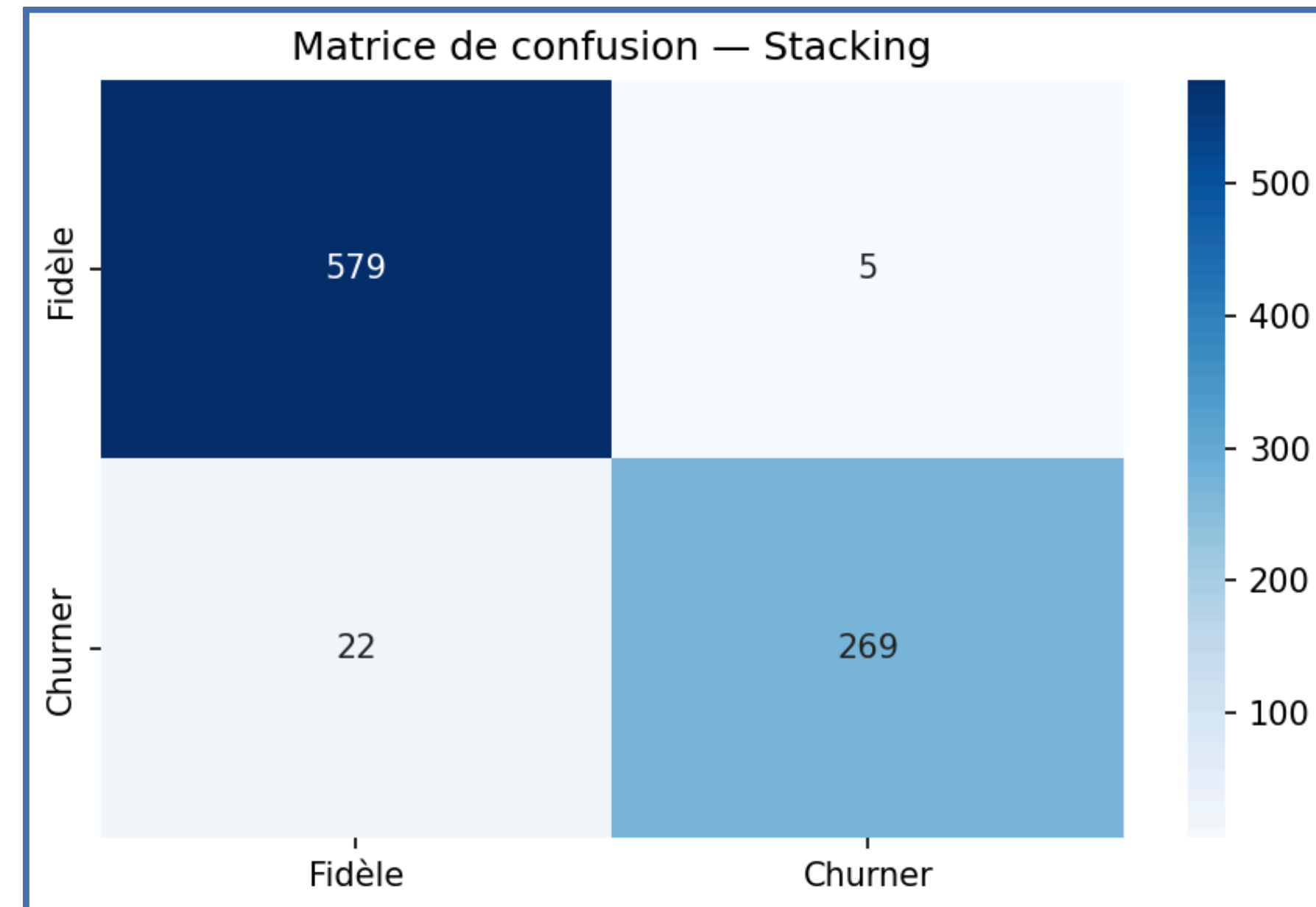
- Niveau 0 : RF + XGBoost entraînés sur X_train
- Méta-features : $P(\text{Churn}=1 | x)$ par validation croisée (5 plis)
- Niveau 1 : Régression logistique sur méta-features

Coefficients méta-modèle :

- XGBoost : 5,82 (naturellement surpondéré)
- Random Forest : 2,52
-

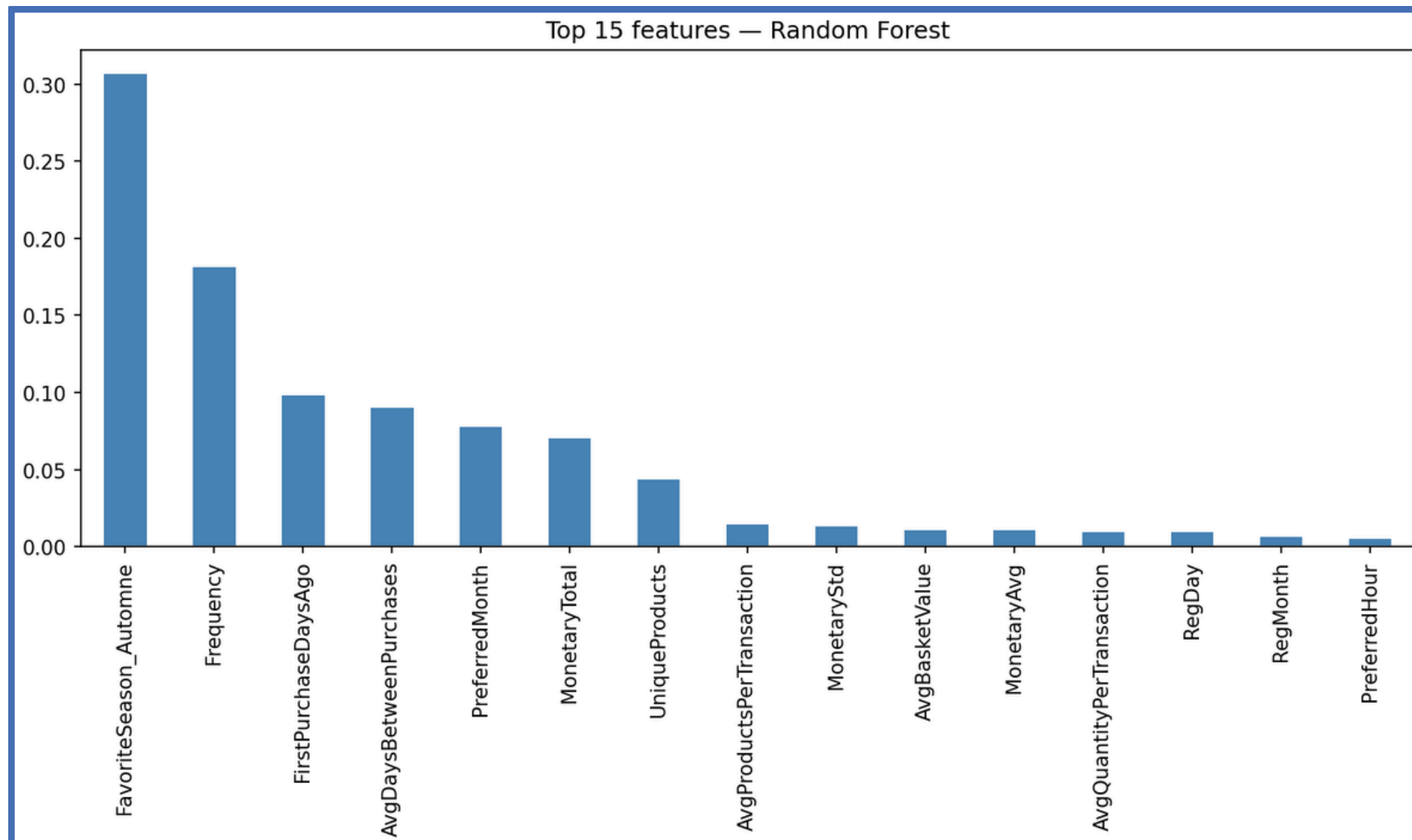
Résultats (seuil=0,50) :

- Fidèle (0) : $P=0,97$ | $R=0,99$ | $F1=0,98$
- Churner (1) : $P=0,98$ | $R=0,93$ | $F1=0,95$
- **Accuracy = 97,0% | AUC = 0,993**

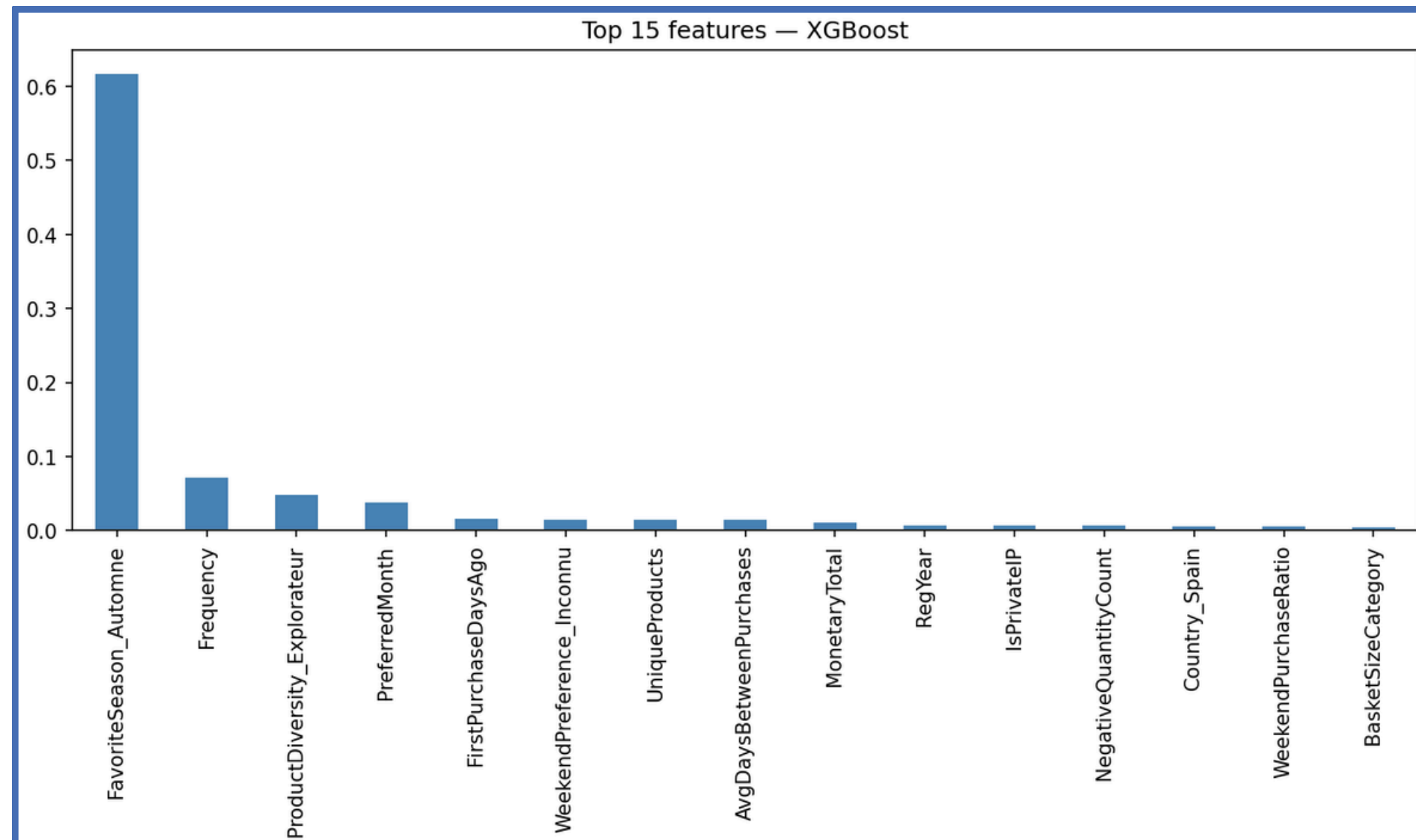


05-Importance des Features

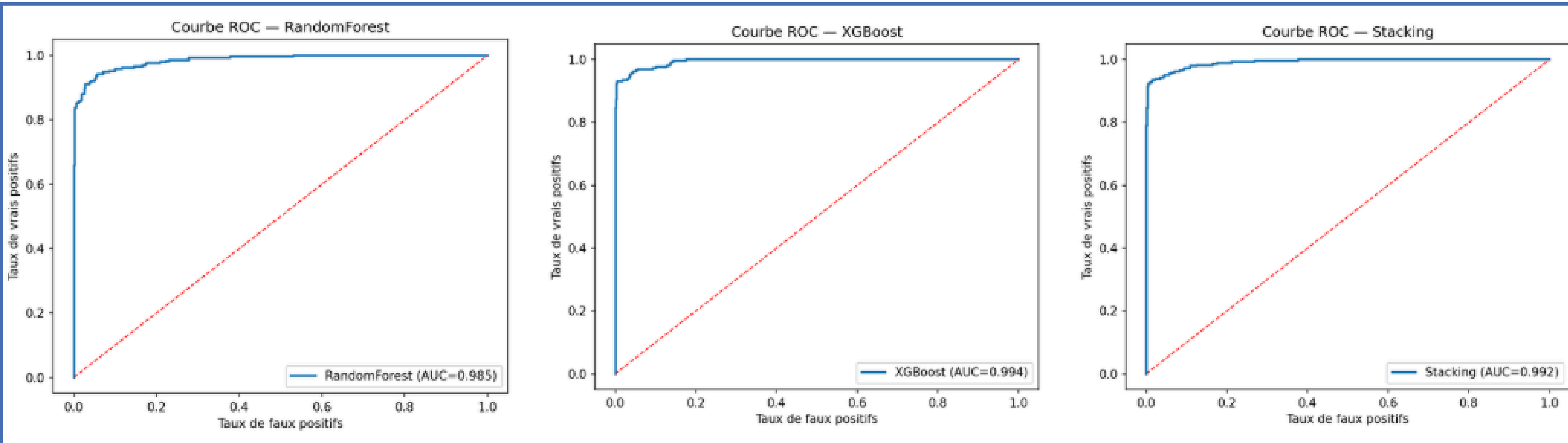
Top 15 Features — Random Forest



Top 15 Features — XGBoost



05-Courbes ROC comparatives



AUC Random Forest: 0,985

AUC XGBoost ★: 0,994

AUC Stacking: 0,993

06 — Régression & Évaluation Comparative

Prédiction du Montant Total Dépensé (MonetaryTotal)

Métrique	Régression Linéaire	XGBoost Regressor
R ² (coef. de détermination)	0,390	0,545 (+39,7%)
RMSE (erreur quadratique)	8 860 £	7 648 £ (↓13,7%)
MAE (erreur absolue)	6 450 £	5 520 £ (↓14,4%)

Note : R²=0,545 reste insuffisant pour une production critique , distribution très asymétrique de MonetaryTotal.

06 — Régression & Évaluation Comparative

Comparaison Globale : Classification Churn

Modèle	Accuracy	F1 Churner	Rappel Churn	AUC-ROC	Seuil
Random Forest + SMOTE	94,5%	0,92	0,89	0,985	0,47
XGBoost + SMOTE ★	97,1%	0,96	0,92	0,994	0,60
Stacking RF + XGB	97,0%	0,95	0,93	0,993	0,50

- Impact correction data leakage : Random Forest 100% → 94,5%
- La chute n'est pas une dégradation : c'est une correction indispensable.

07-Déploiement : Application Flask

Architecture et Flux de Traitement

5 étapes du flux :

- ① Saisie via formulaire HTML (4 champs clés)
- ② Flask reçoit la requête HTTP POST
- ③ Standardisation (scaler.pkl) + Imputation (median_train.pkl)
- ④ Prédiction par les 5 modèles :
random_forest.pkl, xgboost.pkl, stacking.pkl, kmeans.pkl, regression_xgboost.pkl
- ⑤ Résultats injectés dans index.html → navigateur

Modèles chargés :

- kmeans.pkl, pca.pkl, scaler.pkl, ohe_country.pkl, median_train.pkl, random_forest.pkl, xgboost.pkl, stacking.pkl, regression_xgboost_optimized.pkl, scaler_regression.pkl

07-Déploiement : Application Flask

Fonctionnalités de l'Interface

PRÉDICTION	Churn (Fidèle/Churner) Pour chacun des 3 classifieurs avec probabilités associées	VALEUR	Montant Total Prédit Estimation par le XGBoost Regressor (£)
SEGMENT	Cluster K-Means (A/B/C/D) Appartenance au cluster + description comportementale	DASHBOARD	Dashboard Analytique KPIs, comparaison modèles, variance ACP, profils segments

07-Conclusion & Perspectives

Contributions Principales

Data Leakage

Détection & correction (100% → 94,5%)

Classification

XGBoost : 97,1% accuracy, AUC=0,994

Régression

R^2 amélioré 0,39→0,545 (+39,7%)

Segmentation

4 clusters, silhouette +127% avec ACP

Limites et Perspectives

- Silhouette K-Means modeste (0,086) : clusters peu séparés
- $R^2=0,545$ insuffisant pour production critique
- Dataset synthétique — à tester sur données réelles
- → LightGBM, TabNet, Docker, MLflow, DVC

Merci pour votre attention !